

Resumo simples da nova adição na API

A API recebeu uma nova funcionalidade de upload de imagens para os produtos do restaurante. Agora, ao cadastrar um prato ou bebida, também é possível enviar uma foto junto dos dados do produto.

Para implementar isso, foi utilizada a biblioteca **Multer**, uma tecnologia muito usada no ecossistema Node.js para lidar com envio de arquivos em aplicações Express.

O Multer funciona como um **middleware**. Isso significa que ele intercepta a requisição antes dela chegar ao controller. Durante esse processo, ele recebe o arquivo enviado pelo frontend, valida o tipo da imagem, define o nome final do arquivo e salva a imagem na pasta do projeto.

O envio da imagem é feito utilizando o formato **multipart/form-data**, diferente do tradicional `application/json`. Esse formato permite enviar arquivos e textos juntos na mesma requisição, por isso o nome, `multipart`, pois são múltiplas partes na requisição. Além disso, com o arquivo, já não é possível fazer uma requisição apenas com JSON.

Também foram utilizados módulos nativos do Node.js:

- `fs`: utilizado para criar pastas automaticamente e remover arquivos em caso de erro;
- `path`: utilizado para criar caminhos seguros entre diferentes sistemas operacionais.

A arquitetura MVC da aplicação foi mantida durante toda a implementação:

- **Routes**: definem as rotas e aplicam o middleware do Multer;
- **Controller**: recebe os dados da requisição e monta o objeto do produto;
- **Service**: valida os dados do produto;
- **Repository**: salva as informações no MySQL.

A imagem é salva fisicamente na pasta:

`public/uploads/produtos`

Enquanto isso, o banco de dados armazena apenas o caminho da imagem:

/uploads/produtos/nome-da-imagem.png

Isso deixa a aplicação mais leve e organizada, já que o banco não precisa armazenar arquivos binários diretamente.

Também foi configurado o **express.static**, permitindo que as imagens possam ser acessadas pelo navegador através de URLs públicas.

O que essa funcionalidade trouxe

- cadastro de produtos com imagem;
- organização de uploads no servidor;
- validação de formatos de imagem;
- separação entre arquivos físicos e banco de dados;
- manutenção da arquitetura MVC;
- tratamento de erros e remoção automática de arquivos órfãos.